Deployment of Education App to Kubernetes

Course Name: DevOps

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Sr.no	Student Name	Enrolment Number
1	ABHIKET THAKUR	EN22CS301030
2	ADITYA PALIWAL	EN22CS301055
3	AARUSHI MITTAL	EN24CS3T10026
4	AKASH KUSHWAHA	EN22CS301083
5	AKSHAY DASHORE	EN22CS301096
6	AKSHITA RATHORE	EN22CS301100

Institution Name: Medi-caps University – Datagami Skill Based Course

Group: 04D1

Project No: GAI-04

Industry Mentor Name: Aashruti Shah

University Mentor Name: Dr. Ritesh Joshi

Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

Jan - May 2026

1. Problem Statement & Objectives

1.1 Problem Statement

With the increasing demand for online education platforms, there is a need for applications that are not only user-friendly but also highly available, scalable, and easy to maintain. Traditional deployment methods often face challenges such as downtime during updates, limited scalability, and inefficient resource utilization.

In this project, the problem is to design and implement a reliable deployment strategy for an education web application that ensures continuous availability and efficient management of application resources. The system should be capable of handling multiple users, supporting seamless updates, and minimizing service interruptions.

To address these challenges, the project focuses on containerizing a React-based Education application and deploying it on a Kubernetes cluster using Minikube. It also aims to implement zero-downtime rolling updates and integrate CI/CD pipelines to automate the build and deployment process.The objective is to provide a scalable, efficient, and modern deployment solution that overcomes the limitations of traditional approaches and ensures a smooth user experience.

1.2 Project Objectives

The specific objectives of the project are as follows:

- To develop a responsive Education web application using React.
- To containerize the application using Docker for consistent and portable deployment.
- To deploy the containerized application on a Kubernetes cluster using Minikube.
- To design and configure Kubernetes resources such as Deployments and Services for efficient application management.
- To implement zero-downtime rolling updates for seamless application version upgrades.
- To ensure high availability and fault tolerance by managing multiple application instances (pods).
- To integrate a CI/CD pipeline using GitHub Actions or Jenkins for automated build and deployment.
- To demonstrate modern DevOps practices including containerization, orchestration, and continuous delivery.
- To create a scalable and maintainable deployment architecture for future enhancements.

1.3 Scope of the Project

This project focuses on the deployment of a containerized Education web application using modern DevOps practices. It covers the complete process from application development to containerization and orchestration using Kubernetes.

The scope includes developing a React-based frontend application and packaging it into a Docker container for consistent deployment across environments. The project further involves deploying the application on a Kubernetes cluster using Minikube, where key resources such as Deployments and Services are configured to manage application instances and ensure accessibility.

A major aspect of the project is the implementation of zero-downtime rolling updates, allowing seamless upgrades of the application without interrupting user access. Additionally, the project includes the integration of CI/CD pipelines using tools like GitHub Actions or Jenkins to automate the build and deployment process.

The project demonstrates core concepts such as containerization, orchestration, scalability, and automated deployment. However, the scope is limited to a local Kubernetes environment and does not include full-scale production deployment, advanced security configurations, or complex backend integrations.Future enhancements may include cloud-based deployment, backend service integration, database management, and advanced monitoring solutions.

2. Proposed Solution

The proposed solution aims to address the limitations of traditional deployment methods by adopting a modern, containerized approach. In this project, the Education application is packaged using Docker and deployed on a Kubernetes cluster to ensure scalability, reliability, and efficient resource management.

The solution incorporates Kubernetes features such as Deployments and Services to maintain application availability and handle user requests effectively. Additionally, zero-downtime rolling updates are implemented to allow seamless application upgrades without interrupting user access. Integration of CI/CD pipelines further automates the build and deployment process, resulting in a faster and more reliable system.

2.1 Key features

The project includes several important features that enhance its functionality and usability:

- **Containerized Deployment**: The application is packaged using Docker, ensuring consistency across different environments.
- **Kubernetes Orchestration:** Kubernetes is used to manage, deploy, and scale application containers efficiently.
- **Zero-Downtime Rolling Updates**: Application updates are deployed seamlessly without interrupting user access.
- **High Availability**: Multiple replicas (pods) ensure that the application remains accessible even in case of failures.
- **Scalability**: The system can handle increased user load by scaling the number of pods.
- **CI/CD Integration**: Automated build and deployment using GitHub Actions or Jenkins improves efficiency and reduces manual effort.
- **Load Balancing**: Kubernetes Services distribute incoming traffic evenly across running pods.

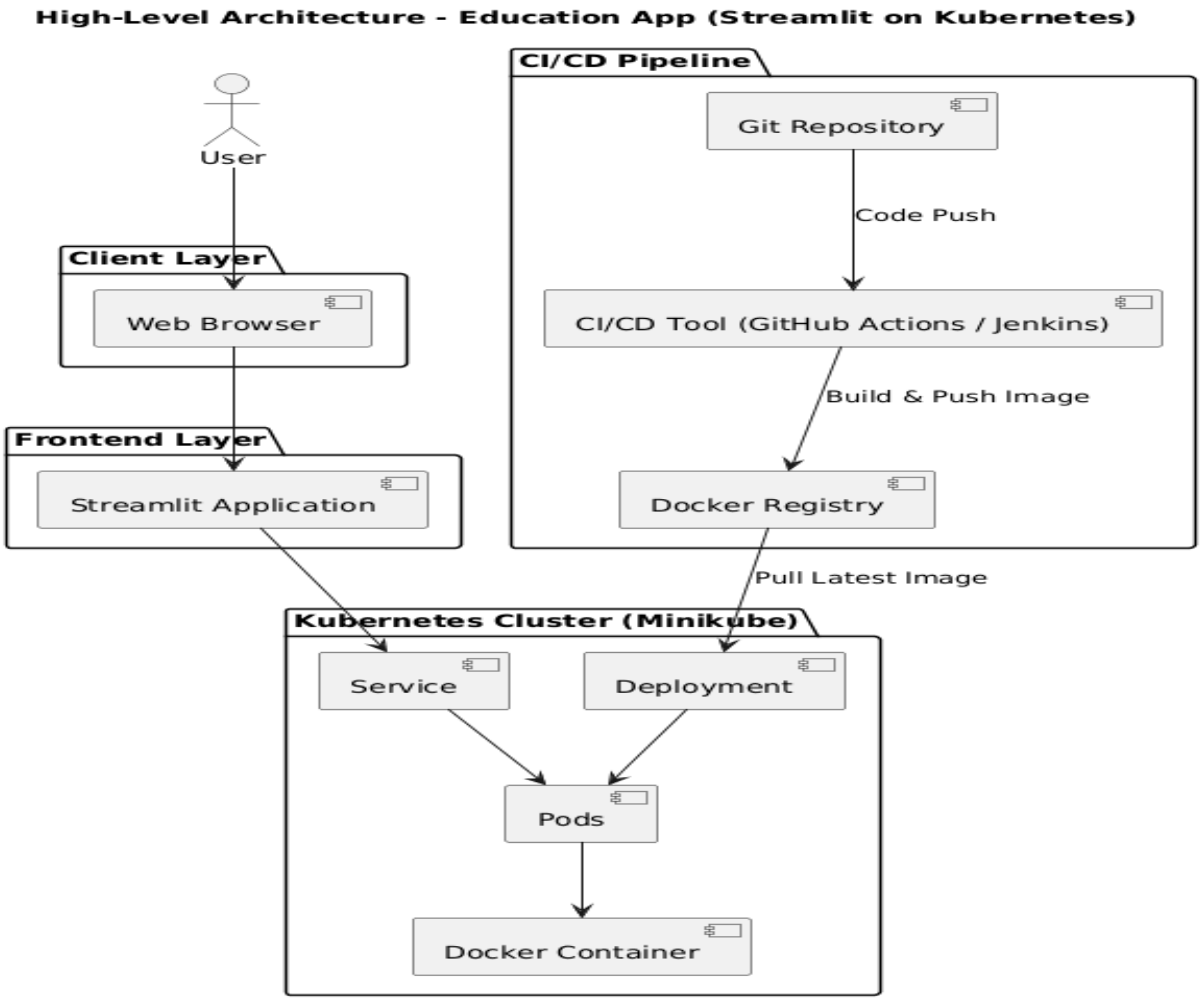
- **Fault Tolerance**: Failed pods are automatically detected and restarted by Kubernetes.
- **Efficient Resource Management**: Optimized utilization of system resources such as CPU and memory.
- **Modular and Maintainable Design**: The system is designed with clear separation of components, making it easy to update and extend.

2.2 Overall Architecture / Workflow

The overall architecture of the system defines how different components interact to deliver a complete deployment solution. It outlines the flow of operations from code development to application deployment and user access.

In this project, the workflow begins with code being pushed to a repository, triggering a CI/CD pipeline that builds a Docker image and stores it in a registry. Kubernetes then pulls the image and deploys it using Deployments and Pods, while Services expose the application to users. The architecture ensures scalability, high availability, and seamless updates through rolling deployment strategies.

High-Level Architecture



The system follows a layered architecture consisting of the Client Layer, Frontend Layer, CI/CD Pipeline, and Kubernetes Cluster.

- The Client Layer includes the user's web browser, through which users access the application.
- The Frontend Layer consists of the Education application, which provides the user interface and handles user interactions.
- The CI/CD Pipeline integrates tools such as Git, GitHub Actions, or Jenkins to automate the process of building Docker images and pushing them to a container registry.
- The Kubernetes Cluster (Minikube) is responsible for orchestrating the deployment. It includes key components such as Deployments, Pods, and Services.

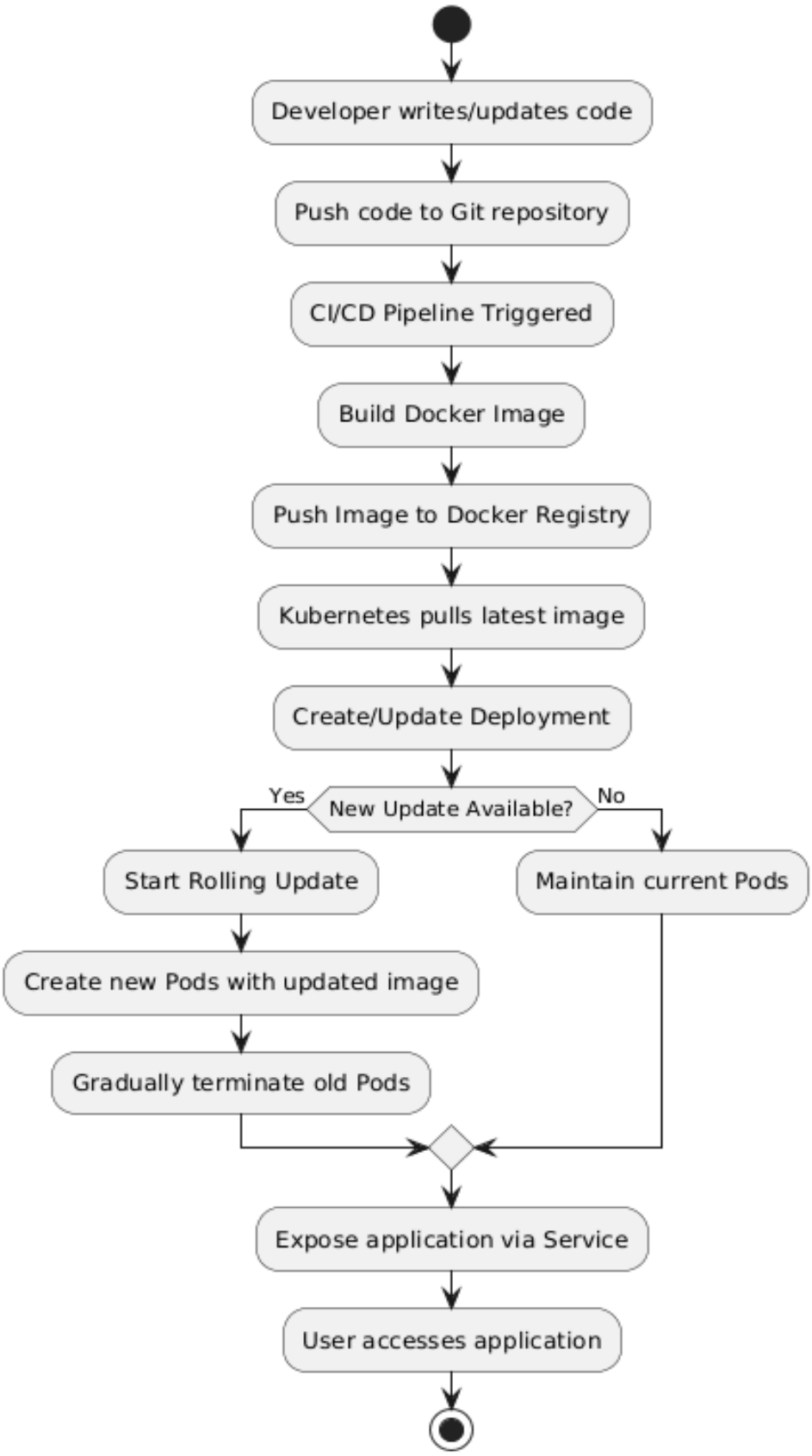
The workflow begins when the developer pushes code to the repository. This triggers the CI/CD pipeline, which builds a Docker image and stores it in a registry. Kubernetes then pulls the latest image and deploys it through a Deployment, which manages multiple Pods running the application. A Service is used to expose the application and distribute incoming user requests across the available pods.

System Workflow

The system workflow describes the step-by-step process involved in developing, deploying, and accessing the Education application using a Kubernetes-based architecture.

1. **Code Development**: The developer builds and updates the React-based Education application on a local system.
2. **Code Push to Repository**: The updated code is pushed to a version control system (Git repository).
3. **CI/CD Pipeline Trigger**: The code push triggers the CI/CD pipeline (GitHub Actions or Jenkins) automatically.
4. **Docker Image Build**: The pipeline builds a Docker image of the application using the provided Dockerfile.
5. **Image Push to Registry**: The built Docker image is pushed to a container registry for storage and access.
6. **Kubernetes Deployment Update**: Kubernetes pulls the latest Docker image from the registry and updates the Deployment configuration.
7. **Pod Creation and Management**: The Deployment creates and manages multiple Pods, each running a containerized instance of the application.
8. **Service Exposure**: A Kubernetes Service exposes the application and routes incoming user requests to the available pods.
9. **User Access**: Users access the application through a web browser, and requests are handled by the Service and Pods.
10. **Rolling Updates**: When a new version is deployed, Kubernetes performs rolling updates to replace old pods with new ones without downtime.

Process Flow - Education App Deployment on Kubernetes



2.3 Tools and Technologies Used

1. Streamlit

- **Streamlit** is an open-source Python framework used to quickly build and deploy interactive web applications, especially for data science and machine learning projects.
- It allows developers to create apps with simple Python scripts without needing frontend development skills.

2. Git

- Used for version control and source code management
- Enables tracking of code changes and collaboration

3. Docker

- Used for containerizing the application
- Ensures consistency across different environments
- Packages application along with its dependencies

4. Kubernetes (K8s)

- Used for container orchestration and management
- Handles deployment, scaling, and monitoring of containers
- Ensures high availability and fault tolerance

5. AWS

- **Amazon Web Services (AWS)** is a cloud computing platform that provides on-demand services like computing power, storage, and databases over the internet.
- It allows developers to build, deploy, and scale applications without managing physical infrastructure.

6. CI/CD Tools (GitHub Actions / Jenkins)

- Automates the build, test, and deployment process
- Reduces manual effort and improves reliability
- Enables continuous integration and delivery

7. Docker Registry (Docker Hub)

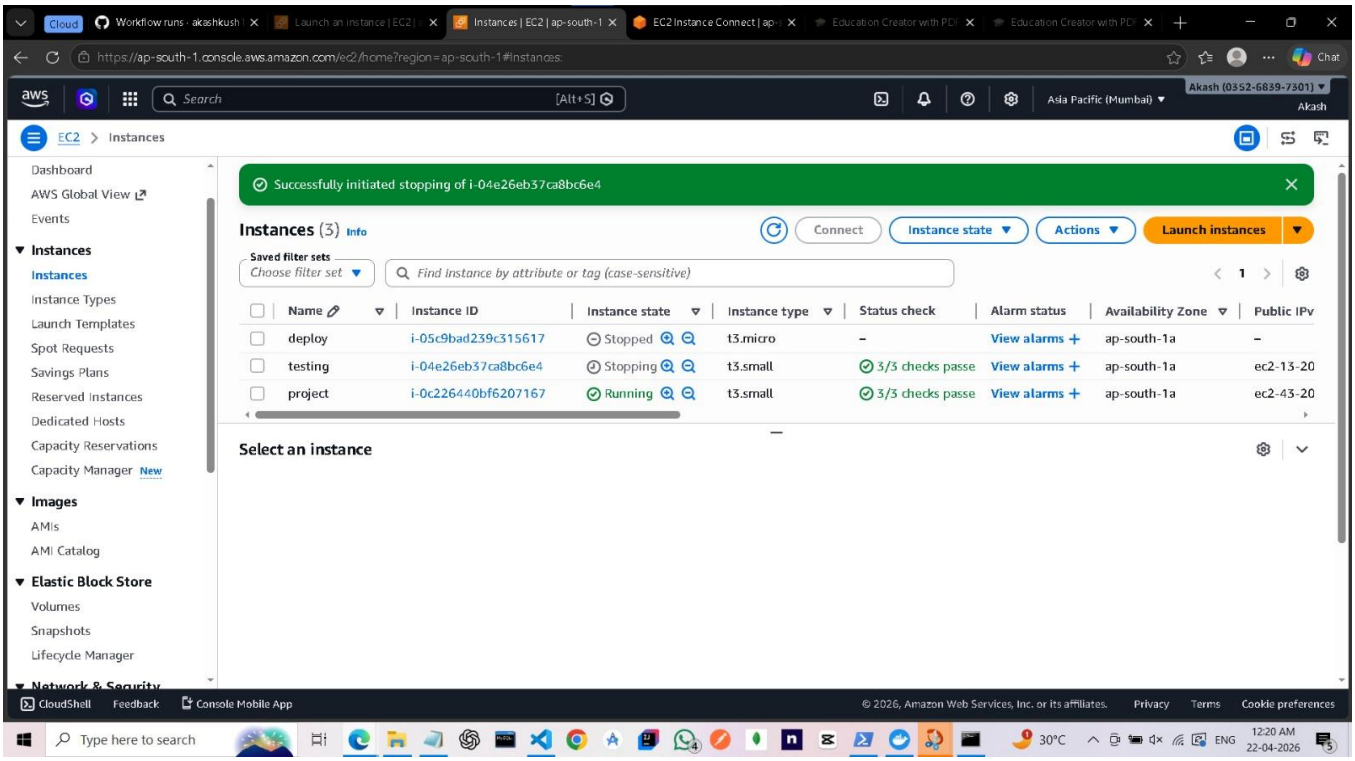
- Stores Docker images for deployment
- Allows Kubernetes to pull the latest application image

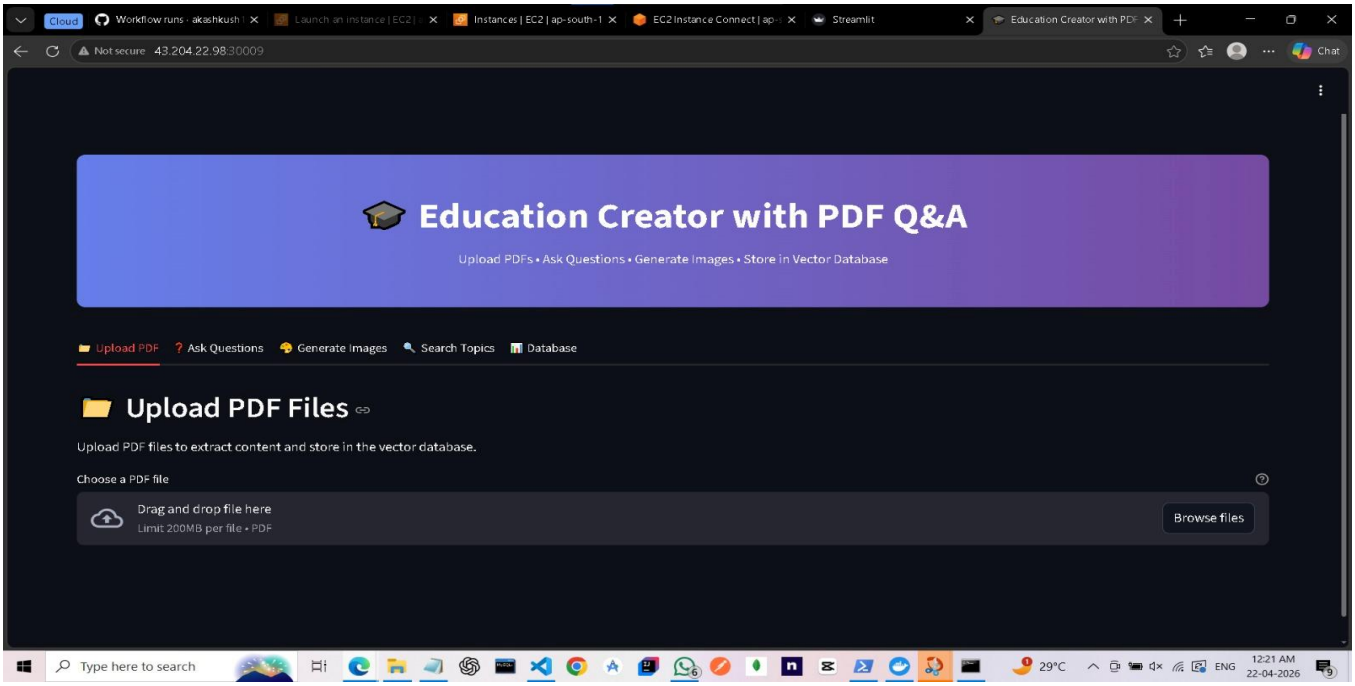
8. YAML

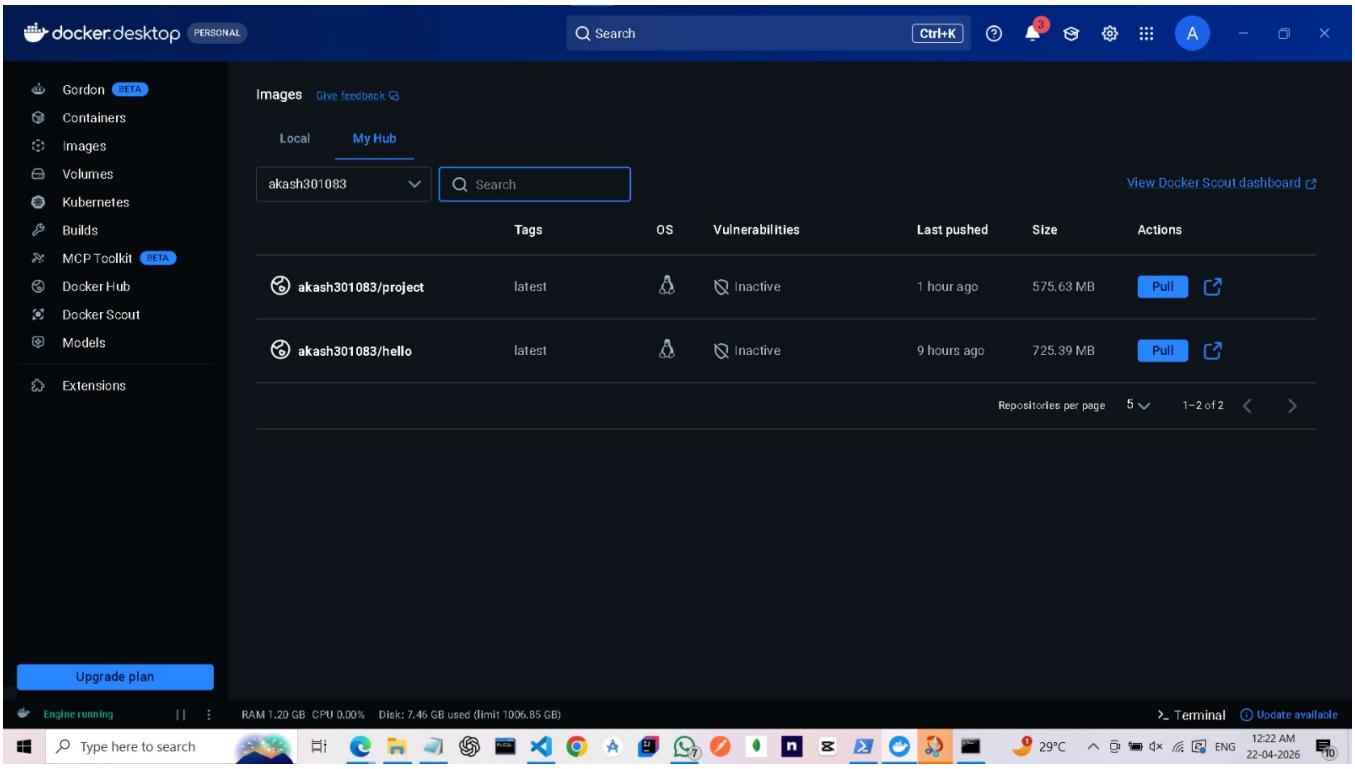
- Used for writing Kubernetes configuration files
- Defines Deployments, Services, and other resources

3. Results & Output

3.1 Screenshots / outputs







3.2 Reports / Dashboards / Models

Reports, dashboards, and models provide insights into the performance and behavior of the system. They help in monitoring system activities, analyzing usage patterns, and evaluating overall efficiency.

In this project, basic reporting and monitoring can include information such as the number of deployments, system status, and application performance metrics. Dashboards can be used to visualize key data like active pods, resource usage, and response times. Models represent the structure and workflow of the system, helping in better understanding and documentation of the deployment process.

System Reports

The system can generate or maintain basic reports such as:

1. Deployment Reports

- Tracks details of application deployments
- Includes information about deployment versions and update history
- Helps in monitoring rolling updates and changes

2. System Performance Reports

- Provides data on response time and system performance
- Helps in analyzing application efficiency under different loads

3. Resource Utilization Reports

- Monitors CPU and memory usage of pods
- Helps in optimizing resource allocation within the Kubernetes cluster

4. Error and Log Reports

- Captures application errors and system logs
- Useful for debugging and troubleshooting issues

5. Usage Reports

- Tracks user interactions with the application
- Includes information such as number of users and requests handled

6. CI/CD Pipeline Reports

- Provides details about build and deployment status
- Includes success/failure logs of pipeline executions

7. Health and Status Reports

- Shows the current status of pods and services
- Helps in ensuring system availability and reliability

3.3 Key outcomes

The successful completion of this project resulted in the following key outcomes:

- **Successful Deployment on Kubernetes**: The Education application was successfully deployed on a Kubernetes cluster using Minikube.
- **Implementation of Containerization**: The application was containerized using Docker, ensuring consistency and portability across environments.
- **Zero-Downtime Rolling Updates**: Achieved seamless application updates without interrupting user access using Kubernetes rolling update strategy.
- **High Availability and Reliability**: Multiple pods ensured continuous availability of the application even in case of failures.
- **Automated CI/CD Pipeline**: Integrated CI/CD tools (GitHub Actions/Jenkins) to automate the build and deployment process.
- **Efficient Resource Management**: Kubernetes effectively managed system resources such as CPU and memory.
- **Improved Scalability**: The system design supports scaling of application instances based on demand.

- **Enhanced Understanding of DevOps Practices**: Gained practical experience in modern tools like Docker, Kubernetes, and CI/CD pipelines.
- **Modular and Maintainable System Design**: The architecture supports easy updates and future enhancements.

4. Conclusion

This project successfully demonstrates the deployment of a containerized Education web application using Kubernetes. By integrating modern tools such as Docker, Kubernetes, and CI/CD pipelines, the system achieves efficient, scalable, and reliable application management. The implementation of Kubernetes Deployments and Services ensures high availability and fault tolerance, while the use of rolling updates enables zero-downtime application upgrades.

The automated CI/CD pipeline further enhances the efficiency of the development and deployment process by reducing manual intervention. Overall, the project highlights the importance of DevOps practices in modern application development and deployment. It provides a strong foundation for building scalable and production-ready systems. Future enhancements can include cloud-based deployment, backend integration, advanced monitoring, and improved security mechanisms.

5. Future Scope & Enhancements

- Cloud Deployment: The application can be deployed on cloud platforms such as AWS, Google Cloud, or Azure for production-level scalability and reliability.
- Backend Integration: A backend service and database can be added to support dynamic data handling, user authentication, and course management.
- Auto-Scaling: Implementation of Kubernetes Horizontal Pod Autoscaler (HPA) to automatically scale pods based on traffic and resource usage.
- Advanced Monitoring and Logging: Integration with monitoring tools like Prometheus and Grafana for real-time system insights and alerting.
- Enhanced Security: Implementation of advanced security measures such as role-based access control (RBAC), secrets management, and network policies.
- Microservices Architecture: Transition from a single application to a microservices-based architecture for better scalability and modularity.

- Use of Helm Charts: Simplify Kubernetes deployments using Helm for better configuration management.
- Content Delivery Network (CDN): Use of CDNs to improve content delivery speed and reduce latency for users.
- AI-Based Features: Integration of recommendation systems, chatbots, or analytics for personalized learning experiences.
- Mobile Application Support: Development of a mobile version of the application for broader accessibility.